

Document number: P1324R0

Date: 2018-10-12

Audience: EWG

Reply-To: Mihail Naydenov <mihailnaydenov at gmail dot com>

RE: Yet another approach for constrained declarations

Abstract

This proposal is direct continuation, a "reply", to [Yet another approach for constrained declarations \(P1141R0\)](#).

It proposes to go "one step further" and allow specifying a type in the place of `auto` in all constrained declarations, suggested by P1141R0. In other words `void sort(Sortable auto& c);` becomes `void sort(Sortable S& c);` and as a result the specified type is introduced in the current scope.

Summary

This proposal is a pure extension, not an alternative or critique of P1141R0.

In a nutshell, allow

```
void f(Sortable auto x);
Sortable auto f();
Sortable auto x = f();
template <Sortable auto N> void f();
```

to also be written as

```
void f(Sortable S x);
Sortable S f();
Sortable S x = f();
template <Sortable S N> void f();
```

Where `S` will be equivalent to a call to `decltype`, given the original P1141R0 declaration, or not using the terse form at all:

```
void f(Sortable auto&& x)
{
    using S = decltype(x);
    // use S
}
template<Sortable S> void f(S&& x)
{
    // use S
}
```

```
void f(Sortable S&& x)
{
    // use S
}
```

Motivation

The main motivation is explored in many other papers, like for example [Concepts in-place syntax \(P0745R0\)](#).

It basically boils down to the fact, it is often just too convenient to have the type be known as it makes the already verbose generic code slightly less so:

```
auto f(Concept auto&& x) { return something(std::forward<decltype(x)>(x)); }, becomes
```

```
auto f(Concept T&& x) { return something(std::forward<T>(x)); }
```

```
[](Number auto x, decltype(x) y) { }, becomes
```

```
[](Number N x, N y) { }
```

And sometimes it is even impossible to create a declaration using the terse form alone:

```
Concept auto f(Number auto a, AnotherConcept</*?decltype(return)?*/> auto b); //< impossible
```

```
Concept T f(Number N a, AnotherConcept<T> U b); //< OK
```

Why this syntax

Preserving an established mental image

The fact that we are dealing with types in their natural form is big advantage in terms of readability and expressiveness.

Compared to P0745R0, we are not introducing additional visual clutter and new constructs to mentally parse and remember.

<pre>void f(Concept{A}&& a, Concept{B}&& b);</pre>	<pre>void f(Concept A&& a, Concept B&& b);</pre>
--	--

Compared to the original constrained `auto` proposal, we are not only gaining a declared type, but we also keep the declaration familiar-yet-different to standard declarations, so much so that even *unconstrained* declaration are better served.

<pre>void f(Concept auto&& frwref, C&& rvalue);</pre>	<pre>void f(Concept A&& frwref, C&& rvalue);</pre>
<pre>void f(auto& a, auto& b);</pre>	<pre>void f(Any T& a, Any U& b); //< explicit and expressive</pre>

In the unconstrained example, the code is also ready to be "upgraded" to using a more constraining concept without any loss of clarity, visual similarity or the need of additional declarations and/or expressions.

Not only that, the developer can track uses of "loose typing" by searching for all uses of the `Any` concept.

Notice also how the code is much more familiar to the established `template<class T, class U> void f(T& a, U& b);`, *both* by the fact we have named types *and* the fact we actually can use a custom, one letter name - something that would be lost if we were allowed to write the tersest form `void f(Any& a, Any& b);` //< which any? `std::any` or the `Any` concept?

Preserving established declaration rules

Consider what an `int n` states, read right to left.

"An object named 'n' of type integer."

But what is a type here? A type is representation in memory, technically, but a *constraint* semantically - the value of an `int` object can only be in certain range and only whole numbers. One can say, the value of a variable is constrained by its type - in a way, we have been writing constraints for decades, constraints on values. Let's rephrase the above with that in mind:

"Object named 'n', that can have [only] values, belonging to the set of the `int` type."

Why should constraints on types be expressed much differently?

Consider now `Number Num n`:

"Object named 'n', that can have [only] values, belonging to the set named `Num`, `Num` can [only] be a type, belonging to the set of the `Numbers` concept."

As You can see, we are not inventing anything, we are just recursively applying a constraint, right to left, as we always did. In the case of constrained `auto`, we are simply omitting the name, effectively stating, we don't need it - `Number auto n`.

Sidenote, there are proposals, advertising the possibility to be able to omit the variable name. Using the syntax from [P1110R0](#), we can have fully unnamed, fully unconstrained declaration - `auto _` - alongside the fully named, fully constrained declaration we just described, or any combination b/w named/unnamed variable, named/unnamed type, constrained/unconstrained type and constrained/unconstrained variable.

Preserving established relationship b/w `auto` and a type name

It must be pointed out, `auto` already stands-in for an omitted type! At the moment in all, but one case (structured bindings), if we replace `auto` with a type we end up with a valid declaration, the meaning might change, but the declaration will be valid. This means the user already associates `auto` with a "type stand-in", *even if* the declaration meaning changes by the substituting one with the other.

There is no reason not to continue this relationship - if the user sees an `auto`, he *should* be able to write a type name in its place. What this name represents will be new (a name introduction), but the context is also new, besides we don't stride *too* much away from the established usage..

"OK, but still two declarations in one expression, this is madness!"

Type and variable declaration have always been in C++, inherited from C. And they still have their uses.

```
void process(class C& c);
//...
void something(other)
{
    process(other.get_c());
}
// ...
class C {...};
// ...
void process(C& c)
{
    // definition
}
```

or for example

```
class C
{
    // ...
private:
    const class Helper* sos() const; //< 'Helper' referenced only in just that one place
};
```

As shown, two-in-one declaration is nothing new and although the above expressions can be written separately, this does not change the fact extremely similar syntax, with de facto the same meaning (introduce a type name inline) is already here - we just reinvent it for the new era.

In contrast to the elaborate type specifier the declaration will be limited to the current function declaration, as-if the type was declared as template argument.

It is actually the tersest form!

```
auto copy(InputIterator auto begin, decltype(begin) end, OutputIterator auto out) -> decltype(out);
auto copy(InputIterator{It} begin, It end, OutputIterator{OIt} out) -> OIt;
auto copy(InputIterator It begin, It end, OutputIterator OIt out) -> OIt;
```

Even compared to the original terse syntax

```
auto copy(InputIterator being, decltype(begin) end, OutputIterator out) -> decltype(out);
```

It should be evident, enabling type name introduction in the proposed way, not only does not introduce new constructs and concepts, but serve to reinforce established ones.

Not proposed, but worth mentioning

Having return type name introduction, combined with trailing return type could enable us to have some interesting implications:

```
template<class T>
Container Ret function()
    -> std::map<Key, T, [](const T& a, const T& b){ /*return result*/ }, MyAllocator>
{
    Ret ret;
    // use ret with no repetition
}
```

or for example

```
Number Ret function(Number N a, N b) -> decltype(something(x, b))
{
    Ret ret;
    // use ret with no repetition
}
```

Pretty neat, however there are two problems with that. First, lambdas do not have heading return type, and although this is solvable, the second problem is much more severe - we might want to have two different return types, for example [p1063r0](#) makes use of both of them already.

Related work

Not proposed, but a *possible future extension*, is "forward declared" (and effectively late-evaluated) constrained type name.

```
Concept T;
T val = f();
```

This idea is explored in [Concept-defined placeholder types \(P1158R0\)](#)

Interaction with other proposals

The [Multi-argument constrained-parameter \(P1157R0\)](#) proposal suggest a multi-type constraint in the form

```
template<class A, class B> concept Duo = ...;
template<Duo T U>
void f(T t, U u) {}
```

as an alternative to the introducer syntax (as described in [Function declarations using concepts \(P0694r0\)](#)).

Sadly, this collides with the current proposal as this syntax would mean "value U of type T, constrained by a concept Duo" (and Duo will have to be one template argument concept). There is no resolution to the issue - it will have to be one or the other, though it could be argued having `SomeConcept A B` and `OtherConcept auto C` mean radically different things might be confusing, because, as said, `auto` in general is a stand-in for a type - the user might expect that to be the case here as well!

Conclusion

Allowing concepts to introduce the type they constrain will radically change the the way we write generic code. Doing it so with effectively zero new constructs, reusing established practices, will ease the adoption of the terse form as the preferred one, possibly even for unconstrained cases.